



Experiment-10

Student Name: Deven
Branch: BE CSE
Semester: 5th
Subject Name: Advanced Database
Management System

UID: 24BCF10060

1. Aim/Overview of the practical:

To understand database transactions and the ACID properties, and to implement controlled error handling in PostgreSQL using SAVEPOINTS and exception handling so that a failed insertion (due to invalid or duplicate data) can be undone without losing the other, already-successful insertions carried out in the same transaction.

2. Software/Tools Required:

- PostgreSQL (psql command-line client or pgAdmin)
- A code/SQL editor for writing and running the scripts

3. Theory:

A) What is a Transaction?

A transaction is a sequence of one or more SQL statements that PostgreSQL executes as a single logical unit of work: either every statement in it is applied to the database, or none of them are. A transaction begins with BEGIN and ends with either COMMIT (make the changes permanent) or ROLLBACK (undo everything done since BEGIN).

B) ACID Properties

Property	Meaning
Atomicity	All statements in a transaction are treated as one indivisible unit – either every one succeeds, or (if any one fails) none of them are applied.
Consistency	A transaction can only move the database from one valid state to another; constraints such as UNIQUE and NOT NULL are never left violated once the transaction ends.
Isolation	Concurrent transactions do not see each other's uncommitted, in-progress changes.
Durability	Once a transaction is committed, its effects survive permanently, even after a crash or restart.

C) ROLLBACK and SAVEPOINT

ROLLBACK undoes every change made since the transaction began. A SAVEPOINT marks a named point inside an ongoing transaction; issuing ROLLBACK TO SAVEPOINT <name> undoes only the work carried out after that point, while everything done before it remains intact and the surrounding transaction is not aborted – it can still be committed normally afterwards.

D) Exception Handling in PL/pgSQL

Inside a PL/pgSQL block, a nested BEGIN ... EXCEPTION ... END block behaves the same way as a savepoint: PostgreSQL implicitly wraps it in one, so an error caught by the EXCEPTION clause rolls back only that inner block and not the whole surrounding transaction. This is what allows a sequence of inserts to skip over one bad record and continue with the rest, instead of losing everything already inserted.

Note on the given worksheet example

24BCF10060_Deven

The original worksheet example inserts several student rows inside one transaction but includes no savepoint or exception handling. Because the student name “Anisha” already exists, the duplicate INSERT throws a unique-constraint violation, and PostgreSQL aborts the entire transaction – not just the offending row. This correctly demonstrates atomicity, but it does not satisfy the actual requirement of preserving the valid rows. The three scenarios below first reproduce this behaviour and then show two different ways to fix it using savepoints and exception handling.

4. Steps for Experiment/Practical:

1. Connect to the PostgreSQL database using psql or pgAdmin.
2. Create the students table with a UNIQUE constraint on student_name, and seed it with one existing row (“Anisha”).
3. Scenario A: Run a plain transaction that inserts three rows, one of which duplicates “Anisha”, with no error handling, and observe the entire transaction get rolled back.
4. Scenario B: Re-run the same inserts, placing a SAVEPOINT before each one; when the duplicate insert fails, roll back to just that savepoint, then COMMIT the rest.
5. Scenario C: Wrap each insert in its own PL/pgSQL BEGIN ... EXCEPTION WHEN unique_violation ... END block inside a DO \$\$... \$\$ block, printing a success or failure RAISE NOTICE for each row.
6. Run SELECT * FROM students; after each scenario to confirm which rows were actually kept.

5. Output:

Step 1 – Table setup

```
-- Create the students table with a UNIQUE constraint on the name
CREATE TABLE students (
    student_id    SERIAL PRIMARY KEY,
    student_name  VARCHAR(50) UNIQUE NOT NULL,
    course        VARCHAR(50),
    marks         INT
);

-- Seed one existing record so a later duplicate insert will fail
INSERT INTO students (student_name, course, marks)
VALUES ('Anisha', 'DBMS', 88);
```

Scenario A – naive transaction (no savepoint)

```
BEGIN;

INSERT INTO students (student_name, course, marks) VALUES ('Rohit', 'DBMS',
76);
INSERT INTO students (student_name, course, marks) VALUES ('Meera', 'DBMS',
91);
INSERT INTO students (student_name, course, marks) VALUES ('Anisha', 'DBMS',
65); -- duplicate!

COMMIT;
```

Because there is no savepoint or exception handling, the duplicate-name error aborts the whole transaction. Even though Rohit and Meera appeared to insert successfully, the final COMMIT is silently converted into a ROLLBACK and both rows are lost.

```
adbms=# BEGIN;
BEGIN
```

```

adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Rohit',
'DBMS', 76);
INSERT 0 1
adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Meera',
'DBMS', 91);
INSERT 0 1
adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Anisha',
'DBMS', 65);
ERROR:  duplicate key value violates unique constraint
"students_student_name_key"
DETAIL:  Key (student_name)=(Anisha) already exists.
adbms=# COMMIT;
ROLLBACK  -- transaction was already aborted by the error above
adbms=# SELECT student_name FROM students;
 student_name
-----
Anisha
(1 row)  -- Rohit and Meera were LOST

```

Scenario B – explicit SAVEPOINT

```

BEGIN;

SAVEPOINT sp_rohit;
INSERT INTO students (student_name, course, marks) VALUES ('Rohit', 'DBMS',
76);

SAVEPOINT sp_meera;
INSERT INTO students (student_name, course, marks) VALUES ('Meera', 'DBMS',
91);

SAVEPOINT sp_anisha;
INSERT INTO students (student_name, course, marks) VALUES ('Anisha', 'DBMS',
65); -- fails
ROLLBACK TO SAVEPOINT sp_anisha;  -- undo ONLY this failed insert

COMMIT;

```

Wrapping the risky insert with a SAVEPOINT lets us roll back just that one failed statement. Rohit and Meera remain committed, and only the duplicate Anisha insert is discarded.

```

adbms=# BEGIN;
BEGIN
adbms=# SAVEPOINT sp_rohit;
SAVEPOINT
adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Rohit',
'DBMS', 76);
INSERT 0 1
adbms=# SAVEPOINT sp_meera;
SAVEPOINT
adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Meera',
'DBMS', 91);
INSERT 0 1
adbms=# SAVEPOINT sp_anisha;
SAVEPOINT
adbms=# INSERT INTO students (student_name, course, marks) VALUES ('Anisha',
'DBMS', 65);
ERROR:  duplicate key value violates unique constraint
"students_student_name_key"

```

```

DETAIL:  Key (student_name)=(Anisha) already exists.
adbms=# ROLLBACK TO SAVEPOINT sp_anisha;
ROLLBACK
adbms=# COMMIT;
COMMIT
adbms=# SELECT student_name, marks FROM students ORDER BY student_id;
 student_name | marks
-----+-----
Anisha        |    88
Rohit         |    76
Meera         |    91
(3 rows)  -- Rohit and Meera are PRESERVED

```

Scenario C – PL/pgSQL exception handling

```

DO $$
BEGIN
    BEGIN
        INSERT INTO students (student_name, course, marks) VALUES ('Karan',
        'DBMS', 80);
        RAISE NOTICE 'SUCCESS: Karan inserted';
        EXCEPTION WHEN unique_violation THEN
            RAISE NOTICE 'FAILED: Karan - duplicate student name';
        END;

        BEGIN
            INSERT INTO students (student_name, course, marks) VALUES ('Anisha',
            'DBMS', 70);
            RAISE NOTICE 'SUCCESS: Anisha inserted';
            EXCEPTION WHEN unique_violation THEN
                RAISE NOTICE 'FAILED: Anisha - duplicate student name already
exists';
            END;

            BEGIN
                INSERT INTO students (student_name, course, marks) VALUES ('Divya',
                'DBMS', 85);
                RAISE NOTICE 'SUCCESS: Divya inserted';
                EXCEPTION WHEN unique_violation THEN
                    RAISE NOTICE 'FAILED: Divya - duplicate student name';
                END;
            END $$;

```

The PL/pgSQL DO block catches the unique_violation exception for each insert individually, prints a clear success/failure message, and continues on to the next statement without losing any previously inserted row.

```

adbms=# DO $$ ... END $$;
NOTICE:  SUCCESS: Karan inserted
NOTICE:  FAILED: Anisha - duplicate student name already exists
NOTICE:  SUCCESS: Divya inserted
DO
adbms=# SELECT student_name, marks FROM students ORDER BY student_id;
 student_name | marks
-----+-----
Anisha        |    88
Rohit         |    76
Meera         |    91
Karan         |    80

```

6. Result

Database transactions and the ACID properties were studied and demonstrated successfully in PostgreSQL. The naive transaction (Scenario A) confirmed atomicity by rolling back all statements when one insert failed. Using SAVEPOINT and ROLLBACK TO SAVEPOINT (Scenario B), only the failed insertion was undone while the valid records were preserved. Finally, PL/pgSQL exception handling with nested BEGIN...EXCEPTION blocks (Scenario C) achieved the same result while also displaying a clear success or failure message for every record.

7. Learning Outcomes

- Understood the concept of a database transaction and the four ACID properties it guarantees.
- Observed atomicity first-hand: an unhandled error inside a transaction rolls back every statement in it, not just the failing one.
- Learned to use SAVEPOINT and ROLLBACK TO SAVEPOINT to undo only part of a transaction while keeping the rest.
- Implemented PostgreSQL exception handling with nested BEGIN ... EXCEPTION ... END blocks inside a DO block, and understood that each such block runs as an implicit savepoint.
- Practiced applying business-logic style error handling (display success/failure per record), similar to PL/SQL exception blocks in other RDBMS systems.